

qa_2

Primero: validar el match exacto en la fecha de observación

```
positive_state_exact_date_qa <- sdf_sql(  
  sc,  
  "  
  WITH positives AS (  
  
    SELECT  
      party_id,  
      observation_date  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
    WHERE label_3m = 1  
  ),  
  
  matched AS (  
  
    SELECT  
      f.party_id,  
      f.observation_date,  
      p.state_code  
  
    FROM positives f  
  
    LEFT JOIN bc3.padd p  
  
      ON f.party_id = p.party_id  
      AND p.data_date = f.observation_date  
  
    WHERE p.data_date >= TO_DATE('2022-01-01')  
      AND p.data_date < TO_DATE('2026-01-01')  
  )  
  
  SELECT  
    COUNT(*) AS joined_rows,  
    COUNT(DISTINCT party_id) AS matched_parties,  
    COUNT(DISTINCT CONCAT(  
      CAST(party_id AS STRING),  
      '|',  
      CAST(observation_date AS STRING)  
    )) AS matched_party_dates  
  
  FROM matched  
  "
```

```
) %>%  
collect()
```

positive_state_exact_date_qa

Pero hay un detalle: al poner el filtro de `p.data_date` en el `WHERE`, el `LEFT JOIN` se convierte en la práctica en `INNER JOIN`. Para saber además cuántos positivos **no hicieron match**, usaría esta versión, que es mejor:

```
positive_state_exact_date_qa <- sdf_sql(  
  sc,  
  "  
  WITH positives AS (  
  
    SELECT  
      party_id,  
      observation_date  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
    WHERE label_3m = 1  
  ),  
  
  padd_filtered AS (  
  
    SELECT  
      party_id,  
      data_date,  
      state_code  
  
    FROM bc3.padd  
  
    WHERE data_date >= TO_DATE('2022-01-01')  
      AND data_date < TO_DATE('2026-01-01')  
  ),  
  
  matched AS (  
  
    SELECT  
      f.party_id,  
      f.observation_date,  
      p.state_code  
  
    FROM positives f  
  
    LEFT JOIN padd_filtered p  
  
      ON f.party_id = p.party_id
```

```

        AND p.data_date = f.observation_date
    )

SELECT
    COUNT(*) AS joined_rows,

    COUNT(DISTINCT party_id)
    AS distinct_parties,

    COUNT(DISTINCT CASE
        WHEN state_code IS NOT NULL
        THEN party_id
    END) AS parties_with_state,

    COUNT(DISTINCT CASE
        WHEN state_code IS NULL
        THEN party_id
    END) AS parties_without_state

FROM matched
"
) %>%
collect()

```

positive_state_exact_date_qa

Esperamos:

distinct_parties = 62,599

Lo importante es ver cuánto vale:

parties_with_state

Si está muy cerca de 62,599, entonces **no necesitamos ninguna lógica de “latest <= observation_date”**. Simplemente usamos el snapshot exacto del mes.

Segundo: validar duplicados únicamente en ese subconjunto

En vez de revisar toda *padd*, hacemos QA solo sobre los positivos y sus observation dates:

```

positive_state_duplicate_qa <- sdf_sql(
  sc,
  "
  WITH positives AS (

```

```

SELECT
    party_id,
    observation_date

FROM bu_frd.model_features_ab_final_2022_2025

WHERE label_3m = 1
)

SELECT
    p.party_id,
    p.data_date,

    COUNT(*) AS rows_per_party_date,
    COUNT(DISTINCT p.state_code) AS distinct_states

FROM positives f

INNER JOIN bc3.padd p

    ON f.party_id = p.party_id
    AND p.data_date = f.observation_date

WHERE p.data_date >= TO_DATE('2022-01-01')
    AND p.data_date < TO_DATE('2026-01-01')

GROUP BY
    p.party_id,
    p.data_date

HAVING COUNT(*) > 1

ORDER BY
    distinct_states DESC,
    rows_per_party_date DESC

LIMIT 100
"
) %>%
collect()

```

positive_state_duplicate_qa

Esto es muchísimo más eficiente porque no agrupa toda `bc3.padd`.

Y un resumen aún más útil

```

positive_state_duplicate_summary <- sdf_sql(
  sc,
  "
  WITH positives AS (

    SELECT
      party_id,
      observation_date

    FROM bu_frd.model_features_ab_final_2022_2025

    WHERE label_3m = 1
  ),

  x AS (

    SELECT
      p.party_id,
      p.data_date,

      COUNT(*) AS rows_per_party_date,
      COUNT(DISTINCT p.state_code) AS distinct_states

    FROM positives f

    INNER JOIN bc3.padd p

      ON f.party_id = p.party_id
      AND p.data_date = f.observation_date

    WHERE p.data_date >= TO_DATE('2022-01-01')
      AND p.data_date < TO_DATE('2026-01-01')

    GROUP BY
      p.party_id,
      p.data_date
  )

  SELECT
    COUNT(*) AS party_date_combinations,

    SUM(
      CASE WHEN rows_per_party_date > 1
        THEN 1 ELSE 0
      END
    ) AS duplicated_party_dates,

    SUM(

```

```

CASE WHEN distinct_states > 1
      THEN 1 ELSE 0
END
) AS party_dates_with_multiple_states,

MAX(rows_per_party_date)
  AS max_rows_same_party_date,

MAX(distinct_states)
  AS max_states_same_party_date

FROM x
"
) %>%
collect()

```

positive_state_duplicate_summary

La métrica crítica es:

party_dates_with_multiple_states

Si da:

0

podemos usar tranquilamente:

```
SELECT DISTINCT party_id, data_date, state_code
```

pero **solo dentro del subconjunto 2022–2025 / positives**, no sobre toda la tabla.

Mi recomendación ahora

No crees `padd_state_clean` sobre toda `bc3.padd`.

Primero prueba el **exact match**:

$$p.data_date = observation_date$$